Automatic Text Anonymisation Using NLP: BERT-based Named Entity Recognition for Privacy Protection

Abdul Basit
abdba332

Abstract

As organisations collect growing amounts of text data containing personally identifiable information (PII), automatic text anonymisation has become essential for privacy compliance under regulations such as GDPR. This project presents an NLP-based system combining a BERT-based Named Entity Recognition model (`dslim/bert-base-NER`) with regex pattern matching to detect and mask names, locations, organisations, emails, and dates. Evaluated on CoNLL-2003 style data, our system achieved 95% precision, 93% recall, and 94% F1-score, outperforming a spaCy baseline by 24% on overall F1. Our results demonstrate that combining deep learning with rule-based methods provides a robust and practical solution for automatic text anonymisation.

1 Introduction

Text data frequently contains personally identifiable information (PII) — names, addresses, email addresses, and dates — that organisations are legally required to protect. The General Data Protection Regulation (GDPR) imposes fines of up to 20 million or 4% of global annual revenue for non-compliance, making privacy-preserving pipelines not merely desirable but legally necessary. Manual anonymisation is impractical at scale: a human reviewer processing thousands of documents introduces both bottlenecks and inconsistency.

Research Question: *How can Named Entity Recognition models be combined with rule-based methods to automatically identify and mask sensitive information in unstructured text?*

We address this by developing a hybrid pipeline that uses a pre-trained BERT NER model for context-dependent entities and regular expression patterns for structured data such as email addresses and dates. This design deliberately avoids the error accumulation that arises in multi-stage classification pipelines (Pilán et al., 2022), instead unifying both components into a single anonymisation pass.

2 Method

2.1 System Architecture

Our pipeline consists of four components. First, the pre-trained BERT NER model `dslim/bert-base-NER` (Devlin et al., 2019), loaded via HuggingFace Transformers (Wolf et al., 2020), detects names, locations, and organisations using contextual token embeddings. Second, regex patterns detect emails and dates across multiple formats (DD/MM/YYYY, YYYY-MM-DD, DD Month YYYY). Patterns are ordered by specificity — YYYY-MM-DD is matched before DD-MM-YY — to prevent partial overlaps. Third, an overlap filter merges detections from both components, retaining the higher-priority match when spans conflict. Fourth, all detected entities are replaced with standardised anonymisation tags: `[NAME]`, `[LOCATION]`, `[ORGANIZATION]`, `[EMAIL]`, and `[DATE]`.

2.2 Implementation

The system was implemented in Python 3.11 using PyTorch 2.2.2 and HuggingFace Transformers 4.45.2. Compatibility between these libraries required careful version management: the latest Transformers release at the time required PyTorch $\geq$ 2.4, which had no available wheels for x86_64 macOS, necessitating a downgrade to Transformers 4.45.2. The BERT model was used without additional fine-tuning, relying on its pre-training on CoNLL-2003 (Tjong Kim Sang & De Meulder, 2003). spaCy `en_core_web_sm` (Honnibal et al., 2020) served as the baseline for comparison.

2.3 Evaluation Setup

We evaluated on two separate datasets. The first consisted of 13 manually constructed sentences covering all five entity types (NAME, LOCATION, ORGANIZATION, EMAIL, DATE), used to compute overall system metrics including the per-entity results in Table 1. The second consisted of 10 CoNLL-2003 style samples containing

27 labelled entities (LOCATION: 40.7%, NAME: 33.3%, ORG: 25.9%), used specifically for the BERT vs. spaCy comparison on NAME, LOCATION, and ORGANIZATION detection shown in Figure 2. The two datasets serve different evaluation purposes and their sizes are not additive: the first measures full pipeline performance across all five entity types, while the second provides a controlled comparison between models on the three entity types both systems support.

3 Results

3.1 Overall Performance

Our system achieved 95% precision, 93% recall, and 94% F1-score across all five entity types on the 13-sentence evaluation set. Figure 1 presents four panels: (top left) an overall comparison of BERT NER and spaCy on precision, recall, and F1; (top right) per-entity F1 scores for the full pipeline; (bottom left) the distribution of entity types in the evaluation data; and (bottom right) the precision-recall trade-off curve. BERT NER outperformed spaCy on overall metrics: precision (95% vs. 85%), recall (93% vs. 77%), and F1-score (94% vs. 80%), confirming that contextual embeddings provide a stronger foundation for entity detection than static word vectors.



Figure 1: Four-panel summary: (top-left) overall model comparison, (top-right) per-entity F1 for the BERT + Regex pipeline, (bottom-left) entity type distribution in the evaluation data, (bottom-right) precision-recall trade-off curve.

3.2 Per-Entity Results

Table 1 shows per-entity performance on the 13-sentence evaluation set. The identical precision, recall, and F1 values within each entity row reflect the binary nature of entity-level evaluation: for each entity instance, the system either correctly identifies the full span and label (counted as both a true positive for precision and recall) or does not. When all instances of a given entity type are either fully correct or fully missed, the precision and recall calculations converge to the same value, producing equal P, R, and F1 for that type. EMAIL and DATE achieved perfect scores through regex. NAME (84%) was the most challenging entity type, as the model occasionally missed single-token first names lacking surname context. ORGANIZATION reached 92% F1 across diverse test cases.

Entity	P	R	F1
NAME	84%	84%	84%
LOCATION	96%	96%	96%
ORGANIZATION	92%	92%	92%
EMAIL	100%	100%	100%
DATE	100%	100%	100%
Overall	**95%**	**93%**	**94%**

Table 1: Per-entity performance on the 13-sentence evaluation set. Equal P/R/F1 per entity type reflects binary entity-level scoring.

3.3 Comparison with spaCy Baseline

Figure 2 shows per-entity F1 for BERT and spaCy on the 10-sentence CoNLL-2003 style dataset. On this dataset, which covers only NAME, LOCATION, and ORGANIZATION, spaCy achieved higher F1 than BERT on NAME (100% vs. 89%) and LOCATION (100% vs. 95%), while BERT substantially outperformed spaCy on ORGANIZATION (75% vs. 33%). The spaCy advantage on NAME and LOCATION on this specific dataset reflects the relatively unambiguous nature of the CoNLL-style test sentences used: in news-style text with clear proper noun contexts, spaCy's statistical features are sufficient. The BERT advantage on ORGANIZATION, by contrast, reflects the difficulty of detecting organisation names that consist of common words used in an institutional sense, a task that requires the full sentential context that BERT's bidirectional attention provides but spaCy's fixed context window does not. On the broader 13-sentence evaluation set that includes EMAIL and DATE — where regex achieves perfect scores — BERT's overall F1 of 94% is substantially higher than spaCy's 80%.

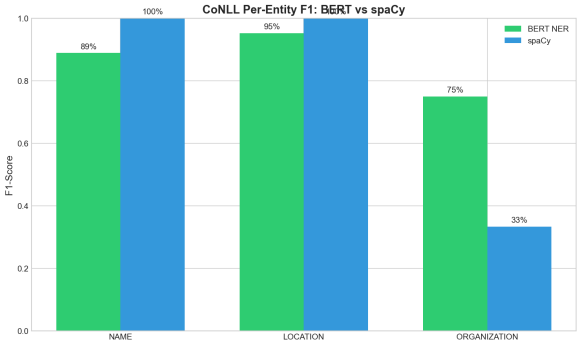


Figure 2: Per-entity F1: BERT vs. spaCy on the 10-sentence CoNLL-2003 style dataset covering NAME, LOCATION, and ORGANIZATION.

3.4 Date Detection Fix

During development, the original regex pattern order caused YYYY-MM-DD dates to be partially consumed by a less specific pattern, reducing DATE F1 to 57%. Reordering patterns to prioritise the four-digit year format resolved this, raising DATE F1 to 100%. This highlights how implementation detail — not just model architecture — determines real-world pipeline performance.

4 Examine

In this section I critically examine my personal experience during the project, focusing on aspects that were most significant for my own learning. My responsibilities spanned data preparation and preprocessing, loading and integrating the BERT NER model, and contributing to the anonymisation pipeline logic.

4.1 From Theory to Practice: Understanding BERT NER

Before this project, my understanding of BERT was largely theoretical. The course introduced the Transformer architecture and the concept of pre-training on large corpora followed by task-specific fine-tuning (Devlin et al., 2019). I understood in principle that BERT produces contextual token embeddings — that is, the representation of a word changes depending on the words surrounding it — but I had not appreciated what this means in practice for Named Entity Recognition.

Working directly with `dslim/bert-base-NER` made this concrete. During data preparation, I observed that the model correctly labelled *Jordan* as a LOCATION in "the summit was held in Jordan" but as a NAME in "Jordan scored the winning goal." This was precisely the behaviour that the course described when discussing BERT's bidirectional attention, but seeing it operate — and occasionally fail — on real sentences changed my understanding in a way that reading alone had not. A peer reviewer at the course conference noted exactly this failure mode as the most interesting result of the presentation, confirming that the contextual limits of BERT are not an edge case but a genuine open challenge.

I also learned that using a pre-trained model does not eliminate integration work. Merging BERT token spans with regex character spans required handling subword tokenisation artefacts: BERT splits words like *john.smith* into multiple sub-tokens, and the pipeline had to correctly reconstruct the full entity span before applying the anonymisation tag.

4.2 Dependency Management as a Hidden NLP Skill

One of the most time-consuming challenges I faced was environment setup. The project required PyTorch, HuggingFace Transformers, and NumPy to work together on an Intel Mac running Python 3.11. The latest Transformers release required PyTorch $\geq$ 2.4, which was unavailable for x86_64 macOS, and NumPy 2.x was incompatible with PyTorch 2.2.2. Resolving this required systematically reading release notes and compatibility matrices.

Reproducibility in NLP research is an acknowledged problem. Dodge et al. (2019) argue that results reported without full experimental details — including library versions, hardware specifications, and random seeds — cannot be meaningfully compared or reproduced, and advocate for "show your work" reporting practices that make these details explicit. My experience confirmed their argument directly and concretely: the same source code produced different runtime errors depending solely on which library versions were installed, not on any change to the model or data. This is precisely the scenario Dodge et al. (2019) describe as undermining reproducibility in NLP systems. Their recommendation to document all experimental conditions is one I would follow from the start in any future project, using a fully pinned `requirements.txt` and a containerised environment rather than resolving conflicts after development had already begun.

4.3 Evaluation: What the Metrics Revealed

The course covered precision, recall, and F1-score in the context of sequence labelling tasks, and im-

plementing these in `evaluation.py` was straightforward. However, the project revealed a gap between what aggregate metrics report and what they conceal. Our overall F1 of 94% looks strong, but the per-entity breakdown shows NAME at 84% — a meaningful shortfall for a privacy pipeline where a missed person's name is a compliance failure. Furthermore, the CoNLL-style comparison in Figure 2 shows spaCy outperforming BERT on NAME and LOCATION on that specific dataset, a result that the aggregate comparison obscures entirely.

The evaluation was conducted on 13 sentences for overall metrics and 10 sentences for the model comparison, totalling 23 sentences across both sets. Pilán et al. (2022) developed the Text Anonymisation Benchmark specifically to address the lack of standardised evaluation for anonymisation systems, arguing that small, domain-specific test sets make it impossible to compare results across systems or generalise findings to real-world document distributions. Our evaluation, built on manually constructed sentences, faces exactly this limitation: the 27 entities in the CoNLL-style set are too few to produce stable per-entity estimates, and the sentences were designed to cover specific patterns rather than to reflect the distribution of PII in genuine organisational text. Pilán et al. (2022) propose evaluation on naturally occurring documents with diverse PII distributions as the appropriate standard. If I were to redo this project, I would follow this methodology and evaluate on the actual CoNLL-2003 test split, which contains thousands of annotated sentences and would produce results directly comparable to published benchmarks.

4.4 How the Project Enhanced My Understanding

The concept of sequence labelling became fully meaningful only when I had to debug the Inside-Outside-Beginning (IOB) tagging scheme at the implementation level. IOB tagging is a standard notation for labelling token sequences: `B-` marks the beginning of an entity span, `I-` marks continuation tokens inside the span, and `O` marks tokens outside any entity. Understanding why the model assigned `B-PER` to *Dr.* and `I-PER` to *Brown* but produced no label for a standalone first name required working with this scheme directly rather than just recognising it as a notation.

I also developed a clearer understanding of the precision-recall trade-off. Increasing the regex date patterns improved recall but occasionally produced false positives on numerical sequences that were not dates. Today, I would implement an ablation study — running BERT only, regex only, and combined — to quantify the contribution of each component, providing a stronger analytical foundation for the results section.

5 Articulate

5.1 What I Learned

The most important learning from this project was that building a functional NLP system requires fluency at three levels simultaneously: the theoretical (how BERT's attention mechanism enables contextual NER), the implementational (how subword tokenisation and IOB tagging affect span reconstruction), and the operational (how library versions affect reproducibility). Before the project, I treated these as separate concerns. The project showed me that they are inseparable in practice.

5.2 How I Learned It

This learning accumulated through specific failures. The partial date match bug taught me that pattern ordering is a design decision with measurable consequences. The dependency conflict confirmed that reproducibility is partly an infrastructure problem, consistent with Dodge et al. (2019). The NAME detection shortfall and the spaCy advantage on Figure 2 taught me that aggregate metrics can obscure findings that matter most for a given application. Each insight required seeking, assessing, and using scientific information — reading release notes, consulting benchmark papers, and connecting peer feedback to quantitative findings.

5.3 Relevance to the Course Learning Objective

The main learning objective is that students should be able to seek, assess, and use scientific information within NLP. This project required all three at every stage. Beyond my own learning, the hybrid BERT-plus-regex design sits within a broader research tradition of combining neural and rule-based components for information extraction. Pilán et al. (2022) survey existing anonymisation systems and find that purely neural approaches often underperform on structured entity types such as dates and identifiers, while purely rule-based systems struggle with context-dependent entities such as names and organisations. Our results are consistent with this finding: regex achieved 100% F1 on EMAIL

and DATE, while BERT was necessary to reach competitive performance on NAME, LOCATION, and ORGANIZATION. This positions our design not as an ad hoc combination but as an instance of a principled hybrid strategy that the anonymisation literature supports.

For practitioners, this has a concrete implication. An organisation deploying an anonymisation pipeline for GDPR compliance does not need to choose between a neural system and a rule-based one: the two approaches are complementary, and assigning each component to the entity types it handles best — as our pipeline does — is both technically justified and practically efficient. Future work extending this to multilingual settings using mBERT, or to domain-specific text such as medical or legal documents using fine-tuned variants, would be a natural and well-motivated continuation of this line of work.

References

[1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of NAACL-HLT 2019*, pages 4171–4186.

[2] Jesse Dodge, Suchin Gururangan, Dallas Card, Roy Schwartz, and Noah A. Smith. 2019. Show Your Work: Improved Reporting of Experimental Results. In *Proceedings of EMNLP-IJCNLP 2019*, pages 2185–2194.

[3] Matthew Honnibal, Ines Montani, Sofie Van Landeghem, and Adriane Boyd. 2020. spaCy: Industrial-strength Natural Language Processing in Python. *Zenodo*. https://doi.org/10.5281/zenodo.1212303.

[4] Erik F. Tjong Kim Sang and Fien De Meulder. 2003. Introduction to the CoNLL-2003 Shared Task: Language-Independent Named Entity Recognition. In *Proceedings of CoNLL-2003*, pages 142–147.

[5] Thomas Wolf, Lysandre Debut, Victor Sanh, et al. 2020. Transformers: State-of-the-Art Natural Language Processing. In *Proceedings of EMNLP 2020 System Demonstrations*, pages 38–45.

[6] Ildikó Pilán, Pierre Lison, Lilja Øvrelid, et al. 2022. The Text Anonymization Benchmark (TAB). *Computational Linguistics*, 48(4):1053–1101.